

TGR-LORA: A CONTINUAL LEARNING PIPELINE FOR SMALL LANGUAGE MODELS

A Design Project Report

Presented to the School of Electrical and Computer Engineering of Cornell
University

in Partial Fulfillment of the Requirements for the Degree of
Master of Engineering, Electrical and Computer Engineering

Submitted by

Beichen Yu (by325)

MEng Field Advisor: Adit Jain (aj457@cornell.edu), Vikram Krishnamurthy
(vikramk@cornell.edu)

Degree Date: December 2026

Abstract

Master of Engineering Program
School of Electrical and Computer Engineering
Cornell University
Design Project Report

Project Title: TGR-LoRA: A CONTINUAL LEARNING PIPELINE FOR SMALL LANGUAGE MODELS

Author: Beichen Yu (by325)

Abstract:

This project implements and evaluates a parameter-efficient (PEFT) continual learning (CL) method designed for small language models (SLMs). It aims to address the problem of catastrophic forgetting, which means during training process across a sequence of different tasks, the parameter changes for learning later tasks might degrade the model’s performance on previous tasks. Our project utilizes the Qwen2.5-3B-Instruct model and conducts experiments using five subsets from the TRACE continual learning enchmark, including C-STANCE, FOMC, ScienceQA, NumGLUE-cm, and NumGLUE-ds.

We implemented two distinct methods. Sequential LoRA serves as our baseline. It freezes the base model parameters and trains a single LoRA adapter across all tasks.

TGR/OrthoHist-LoRA is our method, which trains an active LoRA branch for the current task while integrating and consolidating the updated parameters into a frozen pool of historical bases and coefficients. In order to effectively use historical information, it also learns a low-rank mixing operator (defined as $M_t = I + U_t V_t^\top$) for the current task, thereby enabling the reuse of historical activation information. Furthermore, we implemented the metrics for evaluating continual learning performance, visualizations such as heatmaps showing knowledge reuse degree.

Executive Summary

The main objective of our project was to mitigate catastrophic forgetting while enabling subsequent tasks to leverage useful information acquired from prior tasks. Our system is entirely software-based. The core components include the Qwen2.5-3B-Instruct model, LoRA-based PEFT methods, TGR-LoRA method, and the TRACE benchmark.

Our proposed method involves **training an Active LoRA branch** tailored to the current task. Updated parameters from completed tasks **are stored in a frozen historical pool**, and each new task is equipped with a trainable low-rank mixing operator for reusing these historical activations. During the training process, both the base model and the historical pool remain frozen. There are 2 benefits: preventing the new training process from directly overwriting previously learned directions, while allowing subsequent tasks to reuse knowledge acquired during earlier tasks at the same time. In summary, the design philosophy is that the Active LoRA branch facilitates the model’s exploration of new knowledge of the current task, while the low-rank mixing operator enables the efficient reuse of historical knowledge.

We successfully implemented a comprehensive, task-by-task training loop with checkpoint-based resume, task-level JSON logging, score matrix generation, Backward Transfer (BWT) calculation, average accuracy (AA) computation, and visualizations such as feature reuse heatmaps for the historical mixing operators.

During the project’s initial poster presentation phase, reported results indicated more pronounced positive BWT scores than those subsequently reproduced in the final implementation. During the final code integration and validation phase, we refined our evaluation method, using to a method that computes task-by-task score matrices and BWT metrics.

Individual Contributions

Beichen Yu (by325): This is an individual project. I was responsible for the project’s overall design, code implementation, debugging, experimental execution, poster creation, code repository organization, and final report writing. My specific contributions include:

- Defined the Continual Learning problem setting, selecting the Qwen2.5-3B-Instruct model and the TRACE benchmark as the foundation for experimental validation.
- Implemented a Sequential LoRA baseline model using the PEFT library, freezing the parameters of the Base Model during the implementation process.
- Implemented the TGR-LoRA layer wrapper module. The components include the frozen base model, an active LoRA branch, a frozen pool of historical basis and coefficients, an orthogonal gradient projection mechanism, and a low-rank mixer $M_t = I + U_t V_t^\top$, etc.
- Constructed a Task-by-Task Trainer, checkpoint saving logic, training resume logic, JSON logging function, a score matrix saving function, and a module for analyzing historical knowledge reuse.
- Implemented task-specific answer parsing functions and text generation evaluation metrics for five datasets: C-STANCE, FOMC, ScienceQA, NumGLUE-cm, and NumGLUE-ds to ensure an accurate evaluation method.
- Executed and debugged various experiments within Colab and AutoDL GPU environments, resolving technical challenges such as PyTorch/Transformers compatibility issues, CUDA architecture adaptation problems, and RTX 4090 GPU memory limitations.
- Organized and published the final GitHub code repository including the README and configuration files, and made detailed documentation to ensure the reproducibility of the experiments.

Contents

Abstract	i
Executive Summary	ii
Individual Contributions	iii
1 Problem Statement and Requirements	1
1.1 Design Problem	1
1.2 Requirements	1
1.3 Practical Constraints	2
1.4 Final Design	3
2 Review of Alternatives and Selected Approach	4
2.1 Full Fine-Tuning	4
2.2 Sequential LoRA	4
2.3 Adapter-per-Task	5
2.4 Fisher-SVD Design	5
2.5 Final Selected Approach: TGR/OrthoHist-LoRA	5
3 System Design and Implementation	9
3.1 System Overview	9
3.2 Repository Structure	9
3.3 Sequential LoRA Baseline Implementation	10
3.4 TGR-LoRA Implementation	10
4 Testing and Experimental Validation	12
4.1 Evaluation Method	12
4.2 Sequential LoRA Five-Task Baseline	13
4.3 TGR-LoRA Five-Task Comparison	13
4.3.1 TGR-LoRA with $mt_k = 4$	14
4.3.2 TGR-LoRA with $mt_k = 8$	14
4.3.3 TGR-LoRA with $mt_k = 16$	15
4.3.4 Final Method Comparison	15

Chapter 1

Problem Statement and Requirements

1.1 Design Problem

Continual Learning aims to enable models to train on a continuous stream of incoming tasks. While learning a current task, a model must maintain its performance on previously learned tasks without degradation. Large Language Models (LLMs) are very susceptible to this challenge, as gradient updates applied to subsequent tasks may overwrite specific directions of model parameters that are critical for earlier tasks. The direct consequence of that is Catastrophic Forgetting.

This project focuses on PEFT supervised fine-tuning methods and Small Language Models (SLMs), as these methods are more lightweight and easier to deploy in real-world scenarios. We selected LoRA, as it achieves excellent results by freezing the base model and training only a set of low-rank adapter matrices. However, if a single LoRA adapter is trained sequentially across tasks, the model may still struggle to avoid the forgetting problem. While training a separate adapter for each task effectively prevents interference between tasks, this approach suffers from extremely low parameter reuse. Furthermore, during the inference phase, the corresponding adapter must be manually selected for each specific task, which can be complex to manage. Consequently, the core design challenge of this project is constructing a dynamically growing continual learning system that can both effectively preserve information of historical tasks and enable the efficient reuse of that information.

1.2 Requirements

The final requirements evolved iteratively over the two semesters. The focus of the first semester was to investigate the causes of catastrophic forgetting. Through a comprehensive literature review, we knew that catastrophic forgetting is most severe when the input spaces are similar but the output spaces differ. The work in the second semester focused on designing a robust training and evaluation pipeline with mathematical rigor. The finalized requirements are as follows:

1. **Sequential Task Training:** The system must train tasks sequentially in a fixed order, performing an evaluation after the completion of each task.
2. **Parameter Efficiency:** The Base Model must remain frozen, and only parameters associated with LoRA or TGR-LoRA are permitted to be trained.
3. **Baseline Comparison:** The system must employ Sequential LoRA as a fair comparative baseline.
4. **Historical Memory:** Update information (learned knowledge) generated by completed tasks must be stored within a fixed structure (served as memory).
5. **Historical Reuse:** Subsequent tasks should be able to reuse historical low-rank activations generated by previous tasks via a trainable mixing operator.
6. **Forgetting Metrics:** The system must compute and output a score matrix, average accuracy (AA), and the Backward Transfer (BWT) metric.
7. **Checkpoint Recovery:** If there is an interruption to the Colab or cloud GPU runtime, the training process must be capable of resuming from a task-level checkpoint.
8. **Reproducibility:** The project repository and report must include the complete codebase, execution commands, configuration files, explanations of output results, and environment setup instructions.
9. **Forgetting Mitigation:** The proposed method should effectively reduce the degree of forgetting and achieve a measurable level of Backward Transfer (BWT).

1.3 Practical Constraints

The project faced several constraints, these factors ultimately shaped our design approach.

- **GPU Memory.** The Qwen2.5-3B-Instruct model is substantial in size. Consequently, when performing TGR-LoRA training on an RTX 4090 card with a sequence length of 1024 (model generation) and a historical pool that continuously expands throughout the training process, GPU memory usage can result in an Out of Memory (OOM) error.
- **Cloud Runtime Environment.** Constrained by the resource limits of the Colab and the dynamic variability of the AutoDL environment, as well as the long duration of the training process, the project necessitated the task-level checkpointing to ensure the robustness of the training workflow.
- **Software Compatibility.** Compatibility among PyTorch, Transformers, PEFT, CUDA, and the underlying GPU architecture directly impacted experiments implementation. For instance, the RTX 5090 card requires the use of newer versions of the CUDA and PyTorch software stacks, so we finally choose 4090 for simpler configuration.

- **Evaluation Diversity.** The five TRACE tasks involve distinct types of responses. Therefore, it was necessary to design task-tailored functions for result parsing and the calculation of evaluation metrics.
- **Scope Reduction.** Although the project poster referenced the complete TRACE benchmark dataset, constraints regarding computational resources and time costs ultimately reduced our scope, with the final implementation of five representative tasks.

1.4 Final Design

Design	Final implementation
Backbone model	Qwen2.5-3B-Instruct
Task sequence	C-STANCE, FOMC, ScienceQA, NumGLUE-cm, NumGLUE-ds
Baseline	Sequential LoRA
Proposed method	TGR/OrthoHist-LoRA
Historical memory	Frozen QR-based basis-coefficient pool
Historical reuse	$M_t = I + U_t V_t^\top$ low-rank mixer
Metrics	Task score, average accuracy, BWT, score matrix
Artifacts	<code>score_matrix.png</code> , model checkpoints

Table 1.1: Final design specifications.

Chapter 2

Review of Alternatives and Selected Approach

2.1 Full Fine-Tuning

Full fine-tuning updates all of the model parameters. While this achieves the maximum flexibility, it is computationally expensive and therefore unsuitable for this project. Not only does it necessitate the storage of massive checkpoint files, but is also highly prone to severe catastrophic forgetting. Furthermore, this runs counter to our objective of achieving parameter efficiency.

2.2 Sequential LoRA

LoRA adds a low-rank adaptation branch to the original linear layer. It represents a trainable update to a linear layer as:

$$\Delta W = BA,$$

where $A \in \mathbb{R}^{r \times d_{\text{in}}}$ and $B \in \mathbb{R}^{d_{\text{out}} \times r}$. The forward residual is:

$$\Delta h = BAx.$$

The model’s forward pass for a given input x is represented as the sum of the frozen base layer output and the low-rank adaptation:

$$h = W_0x + \Delta Wx = W_0x + \frac{\alpha}{r}BAx, \quad (2.1)$$

where $W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ is the frozen pre-trained weight matrix, and $\frac{\alpha}{r}$ is a scaling factor defined by the LoRA rank r and the hyperparameter α . During training, only A and B receive gradient updates, while W_0 is frozen.

Sequential LoRA trains one adapter across tasks. It is simple and parameter-efficient, so it is the main baseline. Its weakness is that later task gradients may overwrite the same

LoRA matrices/important directions used by earlier tasks.

2.3 Adapter-per-Task

A separate LoRA adapter could be saved for each specific task. While this method can effectively minimize interference between tasks, it also avoids the sharing of knowledge across them. Furthermore, during the inference phase, this approach needs manual selection of task-specific adapter, which increases the complexity of management.

2.4 Fisher-SVD Design

During the initial phase of design exploration, we designed a method combining Fisher-weighted rank selection with Singular Value Decomposition (SVD)-based basis extraction. Specifically, once a new task is trained, we first employ Fisher information to assess the significance of the active LoRA rank

$$R_j = b_j \otimes a_j^\top = b_j a_j^\top$$

associated with that task. Then, we would utilize Fisher-weighted SVD to compress these critical directions and store them within a historical memory module. Finally, during the training of subsequent tasks, these previously saved directions would be protected. However, this implementation does not achieve a lossless compression of the knowledge acquired during each task, and Fisher information can be accurately estimated only when the model has reached the optimal training state of current task.

2.5 Final Selected Approach: TGR/OrthoHist-LoRA

Our final implementation is designed to mitigate catastrophic forgetting while allowing controlled reuse of historical knowledge. It consists of the following components:

1. **Frozen Base Model:** The original Qwen2.5-3B-Instruct model is frozen. All low-rank adaptation occurs in separate trainable branches.
2. **Active LoRA Branch:** For each new task, a dedicated low-rank adapter is trained. The forward computation for a wrapped linear layer is:

$$h' = W_0 x + \frac{\alpha}{r} (\Delta h_{\text{active}} + \Delta h_{\text{history}})$$

where the active task contribution is

$$\Delta h_{\text{active}} = B_{\text{active}} A_{\text{active}} x.$$

The active branch captures task-specific updates without modifying the frozen base weights.

3. **Historical Basis-Coefficient Pool:** At the end of each task, the active update is committed into a frozen historical pool using QR factorization:

$$B_{\text{active}}A_{\text{active}} = Q(RA_{\text{active}}),$$

where Q (basis) and RA_{active} (coefficients) are stored as historical memory. The coefficients represent how input information is projected into the low-rank feature space (rank-dimension), while the basis maps these features back to the output dimension to modify the model response (output). Both Q and RA_{active} are frozen and can be reused in later tasks. We use QR decomposition to ensure that the model learns different directions/aspects of knowledge for a task, also laying the foundation for gradient orthogonal projection during the training of subsequent tasks.

4. **Low-Rank Historical Reuse:** For a new task t , all previous basis and coefficient blocks are concatenated:

$$B_{\text{old}} = [Q_1, Q_2, \dots, Q_{t-1}], \quad C_{\text{old}} = \begin{bmatrix} C_1 \\ C_2 \\ \vdots \\ C_{t-1} \end{bmatrix}.$$

A trainable low-rank mixing matrix $M_t = I + U_t V_t^\top$ is applied to flexibly combine historical contributions:

$$\Delta h_{\text{history}} = B_{\text{old}} M_t C_{\text{old}} x.$$

This allows the model to reuse previous task knowledge while keeping it separate from the current task updates. mt_k denotes the rank of the mixing matrix M_t , which determines the low-rank dimension for linearly combining historical bases from the Basis Pool. To ensure parameter efficiency, M_t is decomposed into two matrices: $M_{t,U} \in \mathbb{R}^{(r \cdot i) \times mt_k}$ and $M_{t,V} \in \mathbb{R}^{(r \cdot i) \times mt_k}$ (i represents the number of historical tasks). By constraining M_t to a lower rank, the model can combine those features that are most significant from past tasks and possess strong generalization capabilities.

5. **Orthogonal Gradient Hook/Projection:** To prevent interference between new task learning and previously stored historical directions, a backward hook is applied on the active LoRA branch. This hook projects gradients of the active branch onto the subspace orthogonal to the historical basis. So the model’s output direction of the new task is orthogonal to all historical tasks’ output directions. To be specific, if G_{active} is the gradient for the active LoRA B matrix, the hook computes:

$$G_{\text{active}} \leftarrow G_{\text{active}} - Q_{\text{old}}(Q_{\text{old}}^\top G_{\text{active}})$$

where Q_{old} is the concatenated basis from all previous tasks. This ensures that updates

for the current task do not overwrite directions important for historical tasks, effectively reducing catastrophic forgetting.

In summary, our design separates old-task memory from new-task residual learning, allows controlled low-rank reuse, and incorporates orthogonal gradient projection to preserve historical knowledge while adapting to new tasks. During the training process, the base model and historical pool are frozen, while the low-rank mixing matrix and active LoRA are trainable as shown in the following figure.

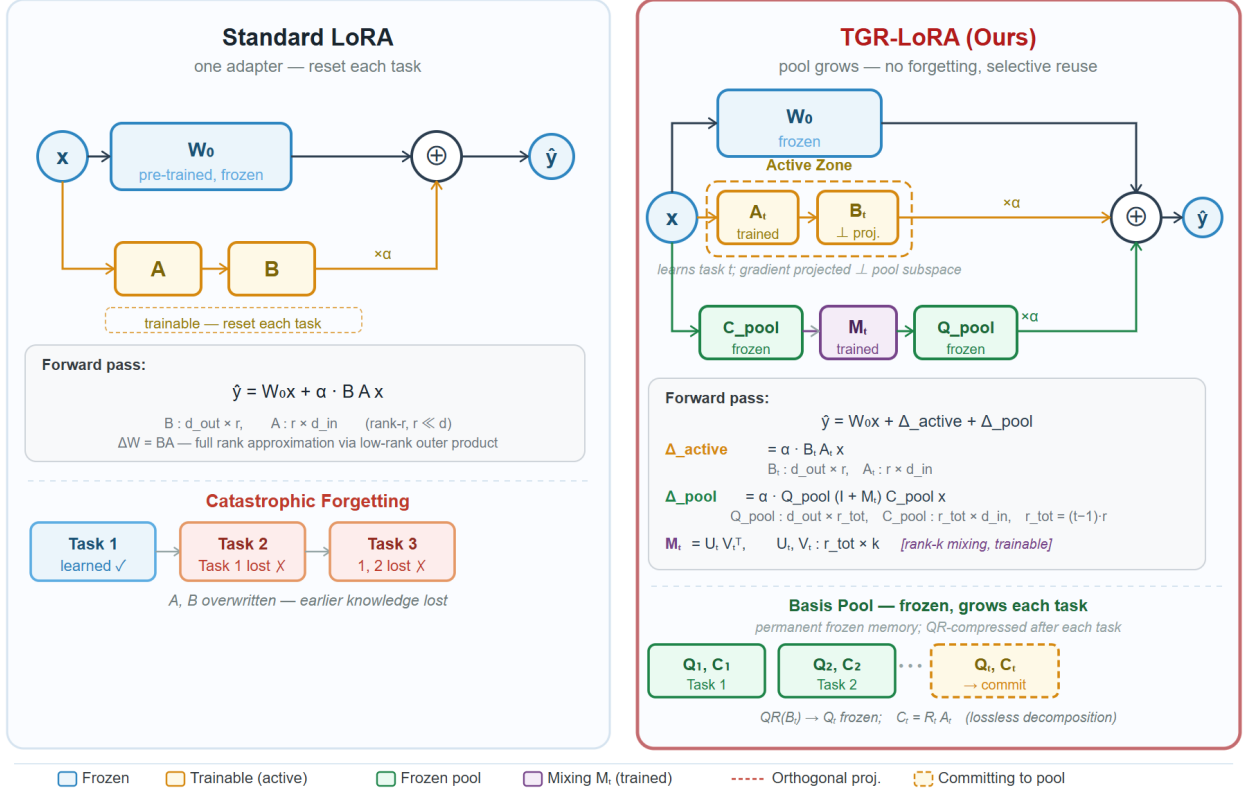


Figure 2.1: Final Design

TGR-LoRA Algorithm

Algorithm 1: TGR-LoRA Continual Learning

Input: Tasks $\{T_1, \dots, T_N\}$; LoRA rank r ; scaling α ; mt-rank k

Output: Model + frozen Basis Pool

```

for each task  $t = 1$  to  $N$  do
  // Step 1 -- Initialize Active Zone
   $A_t \sim \text{Kaiming-uniform}$ 
  if  $t = 1$  then
     $B_t \leftarrow \mathbf{0}$  // standard LoRA cold start
  else
    Sample  $\tilde{B} \sim \mathcal{N}(\mathbf{0}, I)$ 
     $B_t \leftarrow \tilde{B} - Q_{\text{pool}}(Q_{\text{pool}}^\top \tilde{B})$ 
     $B_t \leftarrow \text{QR}(B_t)$  // init orthogonal to all pool bases
  end
  // Step 2 -- Initialize Mixing Matrix  $M_t$ 
   $r_{\text{tot}} \leftarrow (t-1) \cdot r$ 
   $U_t, V_t \in \mathbb{R}^{r_{\text{tot}} \times k}$ , zero-pad from  $M_{t-1}$  (warm start)
  // Step 3 -- Training Loop
  while not converged do
    Forward:  $\hat{y} = W_0 x + \Delta W_t(x)$ 
    Backward, then project gradient:
     $g \leftarrow \nabla_{B_t} \mathcal{L}$ 
    for  $j = 1$  to  $t-1$  do
       $g \leftarrow g - Q_j(Q_j^\top g)$  // Gram-Schmidt onto  $\text{span}(Q_j)^\perp$ 
    end
    Apply projected gradient  $g$  to  $B_t$ 
    Update  $A_t, U_t, V_t$  (no projection)
  end
  // Step 4 -- Commit to Basis Pool
   $[Q_t, R_t] \leftarrow \text{QR}(B_t)$  //  $Q_t^\top Q_t = I$ ; lossless factorisation
   $C_t \leftarrow R_t A_t$  //  $Q_t C_t x = B_t A_t x$  preserved exactly
  Freeze  $(Q_t, C_t) \rightarrow \text{Basis Pool}$ 
  Freeze and archive  $M_t$ 
  Reset:  $B_t \leftarrow \mathbf{0}$ ,
   $A_t \sim \text{Kaiming}$ 
end

```

Figure 2.2: TGR-LoRA Overall Algorithm

Chapter 3

System Design and Implementation

3.1 System Overview

Our project consists of four main components. First, the model and tokenizer are loaded from Hugging Face. Second, PEFT LoRA and TGR-LoRA is injected into selected linear modules (q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj for Qwen-2.5-3B-Instruct). Next, the trainer processes the tasks sequentially, saving a checkpoint and performing an evaluation upon the completion of each task. Finally, the analysis tool saves the score matrix and generates plots.

3.2 Repository Structure

Our repository structure and related content is as follows.

File	Purpose
<code>main.py</code>	Loads configuration, model, tokenizer, and related trainer.
<code>run_experiments.sh</code>	Runs Sequential LoRA and selected TGR-LoRA configurations.
<code>summarise_results.py</code>	Reads run histories and prints summary metrics.
<code>sequential_lora.yaml</code>	Sequential LoRA configuration.
<code>ortho_hist_k4.yaml</code>	TGR-LoRA $mtk = 4$ configuration.
<code>ortho_hist_k8.yaml</code>	TGR-LoRA $mtk = 8$ configuration.
<code>ortho_hist_k16.yaml</code>	TGRTGR-LoRA $mtk = 16$ configuration.
<code>config.py</code>	Experiment Configuration and validation.
<code>data_trace.py</code>	TRACE dataset loading and collation.
<code>eval_trace.py</code>	Task-specific answer parsing and scoring.
<code>trainers.py</code>	TGR-LoRA and Sequential LoRA trainers.
<code>ortho_hist_lora.py</code>	TGR-LoRA layer and manager implementation.
<code>checkpointing.py</code>	Task-level checkpoint saving and loading.
<code>analysis_utils.py</code>	Score matrix and reuse heatmap generation.

Table 3.1: Main repository components.

3.3 Sequential LoRA Baseline Implementation

Sequential LoRA is implemented using the PEFT framework. The base Qwen model remains frozen, with only the LoRA A and B matrices being trainable during the training process. The same LoRA adapter is utilized across all tasks. This serves as the benchmark to assess whether Sequential LoRA suffers from catastrophic forgetting of earlier tasks.

The number of trainable parameters in this benchmark is approximately 29.9 million. This aligns with the parameter count obtained when applying a rank-16 LoRA to the attention projection and MLP projection layers of the Qwen2.5-3B-Instruct model, which validates that the base model freezing logic has been successfully applied.

3.4 TGR-LoRA Implementation

The TGR-LoRA wrapper replaces selected `nn.Linear` modules. Each wrapper stores:

- the frozen base linear layer;
- the active LoRA matrices `lora_A` and `lora_B`;
- a frozen `basis_pool` containing previous basis and coefficient blocks;

- the current task mixer matrices `mt_U` and `mt_V`;
- a backward hook that projects active `lora_B` gradients away from historical basis directions.

At the beginning of task 1, the current active branch is initialized the same as standard LoRA. In subsequent tasks, the active LoRA B is initialized orthogonally to the historical basis pool. Once the training of current task is finished, the current active update will be QR decomposed and store into the historical pool.

Chapter 4

Testing and Experimental Validation

4.1 Evaluation Method

The final implementation evaluates the model after the completion/training of each task. Each row of the score matrix corresponds to the model after the completion of training for a task, while each column corresponds to a task being evaluated. The training sequence for the final five tasks is as follows:

$$\text{C-STANCE} \rightarrow \text{FOMC} \rightarrow \text{ScienceQA} \rightarrow \text{NumGLUE-cm} \rightarrow \text{NumGLUE-ds}.$$

The task evaluation metrics are:

- C-STANCE: accuracy
- FOMC: accuracy
- ScienceQA: accuracy
- NumGLUE-cm: exact match
- NumGLUE-ds: exact match

The continual learning metrics are:

$$AA_t = \frac{1}{t} \sum_{i=1}^t S_{t,i},$$

and

$$\text{BWT}_t = \frac{1}{t-1} \sum_{i=1}^{t-1} (S_{t,i} - S_{i,i}),$$

where $S_{t,i}$ is the score on task i after training through task t .

We choose two classical continual learning metrics. AA_t represents the model’s average performance across all trained tasks after completing the training for the first t tasks. This metric reflects the model’s overall performance as it continuously acquires new knowledge.

BWT_t illustrates the impact of learning new tasks on previously learned tasks. By calculating the difference between a past task’s score at the current moment and its score after the first training, it measures whether the model has suffered from catastrophic forgetting.

4.2 Sequential LoRA Five-Task Baseline

Table 4.1 reports the performance of the Sequential LoRA baseline. The revised results demonstrate that Sequential LoRA serves as a robust baseline in this experiment. Upon completing training for the FOMC task, its C-STANCE score rose slightly from 0.6000 to 0.6100, yielding a corresponding BWT value of +0.0100. Following the subsequent completion of the ScienceQA and NumGLUE-cm tasks, the baseline’s performance remained stable. After finishing the NumGLUE-cm task, its AA score reached 0.7051, with a BWT value of +0.0300.

Upon completing the final task NumGLUE-ds, Sequential LoRA achieved a final AA score of 0.6393 and a final BWT value of -0.0234 , which shows a slight forgetting degree. However, we also note that this outcome is decided according to the experimental configuration: when we set the model’s maximum generation length to just 512 with 500 training samples, the degree of forgetting observed in Sequential LoRA was relatively low. However, when this limit was increased from 512 to 1024, and the number of training samples increased from 500 to 1000, the model exhibited more serious forgetting.

Table 4.1: Sequential LoRA Performance

Trained Up To	C-ST	FOMC	SciQ	Num-cm	Num-ds	AA $\uparrow$	BWT $\uparrow$	FWT $\uparrow$
C-STANCE	0.6000	–	–	–	–	0.6000	0.0000	0.0000
FOMC	0.6100	0.7300	–	–	–	0.6700	+0.0100	0.0000
ScienceQA	0.6000	0.7300	0.8200	–	–	0.7167	0.0000	0.0000
NumGLUE-cm	0.6200	0.7500	0.8700	0.5802	–	0.7051	+0.0300	0.0000
NumGLUE-ds	0.6100	0.7100	0.6500	0.6667	0.5600	0.6393	-0.0234	0.0000

4.3 TGR-LoRA Five-Task Comparison

For TGR-LoRA, we evaluated three settings for the historical mixing rank: $mt_k = 4$, $mt_k = 8$, and $mt_k = 16$. This mixing rank represents the size of the historical reuse operator for each task.

A larger mt_k gives the model greater degrees of freedom to recombine historical low-rank activations, while simultaneously introducing more trainable parameters into the reuse path.

The final results indicate that, in the five-task setting, $mt_k = 8$ achieved the best final AA and BWT scores, yielding a final AA of 0.6296 and a final BWT of -0.0400 . Furthermore, across various configurations, the model’s degree of forgetting consistently followed a pattern of firstly increasing and then decreasing. This trend also persisted after we increased the maximum generation length and the number of training samples. Moreover, under this configuration, the performance gap between TGR-LoRA and Sequential LoRA was smaller.

4.3.1 TGR-LoRA with $mt_k = 4$

Table 4.2 presents the performance of TGR-LoRA under the $mt_k = 4$ configuration. This configuration exhibited the most severe early forgetting phenomenon. Upon completion of the FOMC task, the C-STANCE score dropped from 0.6000 to 0.4400, corresponding to a BWT value of -0.1600 . Subsequent tasks led to a partial recovery of the model’s performance: After completing the ScienceQA task, the BWT improved to -0.0300 , while after completing the NumGLUE-cm task, the BWT briefly turned positive, reaching $+0.0100$. However, upon the completion of the final NumGLUE-ds task, the final AA value was 0.6182, while the final BWT declined to -0.0498 . Therefore, the $mt_k = 4$ configuration failed to provide a stable and effective mitigation of forgetting.

Table 4.2: TGR-LoRA ($mt_k = 4$) Performance

Trained Up To	C-ST	FOMC	SciQ	Num-cm	Num-ds	AA $\uparrow$	BWT $\uparrow$	FWT $\uparrow$
C-STANCE	0.6000	–	–	–	–	0.6000	0.0000	0.0000
FOMC	0.4400	0.7100	–	–	–	0.5750	-0.1600	0.0000
ScienceQA	0.5700	0.6800	0.8200	–	–	0.6900	-0.0300	0.0000
NumGLUE-cm	0.5900	0.7100	0.8600	0.5802	–	0.6851	+0.0100	0.0000
NumGLUE-ds	0.5700	0.7100	0.7000	0.5309	0.5800	0.6182	-0.0498	0.0000

4.3.2 TGR-LoRA with $mt_k = 8$

Table 4.3 presents the performance of TGR-LoRA under the $mt_k = 8$ configuration, which represents the strongest-performing TGR-LoRA configuration. Upon completion of the FOMC task, the C-STANCE score dropped from 0.6000 to 0.4900, yielding a corresponding BWT value of -0.1100 . This result still indicates the early forgetting problem. However, its performance in subsequent stages was more robust than that of the $mt_k = 4$ setting. After completing the ScienceQA task, the BWT improved to -0.0400 . Upon the completion of the NumGLUE-cm task, the BWT further improved to -0.0200 . And after the entire sequence of five tasks, the $mt_k = 8$ configuration finally achieved an AA score of 0.6296 and a BWT value of -0.0400 .

Table 4.3: TGR-LoRA ($mt_k = 8$) Performance

Trained Up To	C-ST	FOMC	SciQ	Num-cm	Num-ds	AA $\uparrow$	BWT $\uparrow$	FWT $\uparrow$
C-STANCE	0.6000	–	–	–	–	0.6000	0.0000	0.0000
FOMC	0.4900	0.7800	–	–	–	0.6350	-0.1100	0.0000
ScienceQA	0.5500	0.7500	0.7900	–	–	0.6967	-0.0400	0.0000
NumGLUE-cm	0.5800	0.7300	0.8000	0.5679	–	0.6695	-0.0200	0.0000
NumGLUE-ds	0.5600	0.7400	0.7100	0.5679	0.5700	0.6296	-0.0400	0.0000

4.3.3 TGR-LoRA with $mt_k = 16$

Table 4.4 reports the performance of TGR-LoRA with $mt_k = 16$. Among all tested configurations, this setting employs the largest historical mixing rank (the same as the LoRA rank 16). During the second-stage tasks, it exhibited less early forgetting compared to the $mt_k = 4$ and $mt_k = 8$ settings. Upon completion of the FOMC task, the C-STANCE metric dropped from 0.6000 to 0.5100, yielding a corresponding BWT value of -0.0900 . However, this advantage did not persist throughout the entire task sequence. After completing the ScienceQA task, the BWT shifted to -0.0750 . Upon finishing the NumGLUE-ds task, the final BWT changed to -0.0687 , which was the weakest final BWT score among all TGR-LoRA configurations. This suggests that simply increasing the mixing rank does not always guarantee improved reuse of historical information.

Table 4.4: TGR-LoRA ($mt_k = 16$) Performance

Trained Up To	C-ST	FOMC	SciQ	Num-cm	Num-ds	AA $\uparrow$	BWT $\uparrow$	FWT $\uparrow$
C-STANCE	0.6000	–	–	–	–	0.6000	0.0000	0.0000
FOMC	0.5100	0.7500	–	–	–	0.6300	-0.0900	0.0000
ScienceQA	0.5100	0.6900	0.8200	–	–	0.6733	-0.0750	0.0000
NumGLUE-cm	0.5500	0.7100	0.8000	0.5802	–	0.6601	-0.0367	0.0000
NumGLUE-ds	0.5600	0.7100	0.6500	0.5556	0.6000	0.6151	-0.0687	0.0000

4.3.4 Final Method Comparison

Table 4.5 shows a comparison of the final performance. Across all completed experiments, Sequential LoRA achieved the best final AA and BWT scores. Among the TGR-LoRA configurations, the configuration with $mt_k = 8$ performed the best, followed by $mt_k = 4$ and $mt_k = 16$.

Consequently, the final results are mixed. Although the TGR-LoRA successfully incorporates historical memory capabilities and trainable reuse of historical information, it currently fails to consistently outperform Sequential LoRA. Furthermore, likely due to the significant difference between the five selected datasets, the current model has only minimal reuse of historical task information at each layer (with values less than 0.001). We also observed that as the number of training samples increases, the performance gap between TGR-LoRA and Sequential LoRA narrows. Therefore, our method appears to be better suited for scenarios of high task similarity and larger training samples, as well as larger model generation length. Future directions should explore the integration of task-aware retrieval or task-specific inference mechanisms to ensure that, when evaluating prior tasks, the system utilizes the historical states most relevant to them.

Table 4.5: Final Comparison After the Five-Task Sequence

Method	C-ST	FOMC	SciQ	Num-cm	Num-ds	Final AA $\uparrow$	Final BWT $\uparrow$
Sequential LoRA	0.6100	0.7100	0.6500	0.6667	0.5600	0.6393	-0.0234
TGR-LoRA ($mt_k = 4$)	0.5700	0.7100	0.7000	0.5309	0.5800	0.6182	-0.0498
TGR-LoRA ($mt_k = 8$)	0.5600	0.7400	0.7100	0.5679	0.5700	0.6296	-0.0400
TGR-LoRA ($mt_k = 16$)	0.5600	0.7100	0.6500	0.5556	0.6000	0.6151	-0.0687